

补充题：

设 $T[1:n]$ 是一个含有 n 个元素的数组。如果元素 x 的出现次数超过 $n/2$ ，称元素 x 为数组 T 的主元素。

(1) 如果这 n 个元素存在序关系，比如 n 个整数

(2) 如果这 n 个元素不存在序关系，比如 n 个坐标

请分别针对上述两种情况，分别设计时间复杂度为 $O(n)$ 的分治算法，判断该数组里是否有主元素。

```

function findMajorityUnordered(nums):
    if nums is empty:
        return False
    candidate, _ = divideAndConquerVote(nums, 0, length(nums) - 1)
    count = 0
    for num in nums:
        if num == candidate:
            count += 1
    return count > length(nums) / 2

function divideAndConquerVote(nums, left, right):
    if left == right:
        return (nums[left], 1) // 只有一个元素, 候选元素是自己

    mid = (left + right) / 2
    (left_cand, left_count) = divideAndConquerVote(nums, left, mid)
    (right_cand, right_count) = divideAndConquerVote(nums, mid + 1, right)

    if left_cand == right_cand:
        return (left_cand, left_count + right_count) // 相同元素, 票数相加

    return (left_cand, left_count - right_count) if left_count > right_count
        else (right_cand, right_count - left_count)

```

$$T(n) = 2T(n/2) + O(1)$$

$O(n)$

时间复杂度: $O(\log n) + O(n) = O(n)$, 空间复杂度 $O(\log n)$

定理 4.1 (主定理) 设 $a \geq 1$ 和 $b > 1$ 为常数, 设 $f(n)$ 为一函数, $T(n)$ 由递归式

$$T(n) = aT(n/b) + f(n)$$

对非负整数定义, 其中 n/b 指 $\lfloor n/b \rfloor$ 或 $\lceil n/b \rceil$ 。那么 $T(n)$ 可能有如下的渐近界:

1) 若对于某常数 $\epsilon > 0$, 有 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$;

4-1 会场安排问题。



问题描述：假设要在足够多的会场里安排一批活动，并希望使用尽可能少的会场。设计一个有效的贪心算法进行安排。（这个问题实际上是著名的图着色问题。若将每个活动作为图的一个顶点，不相容活动间用边相连。使相邻顶点着有不同颜色的最小着色数，相当于要找的最小会场数。）

算法设计：对于给定的 k 个待安排的活动，计算使用最少会场的时间表。

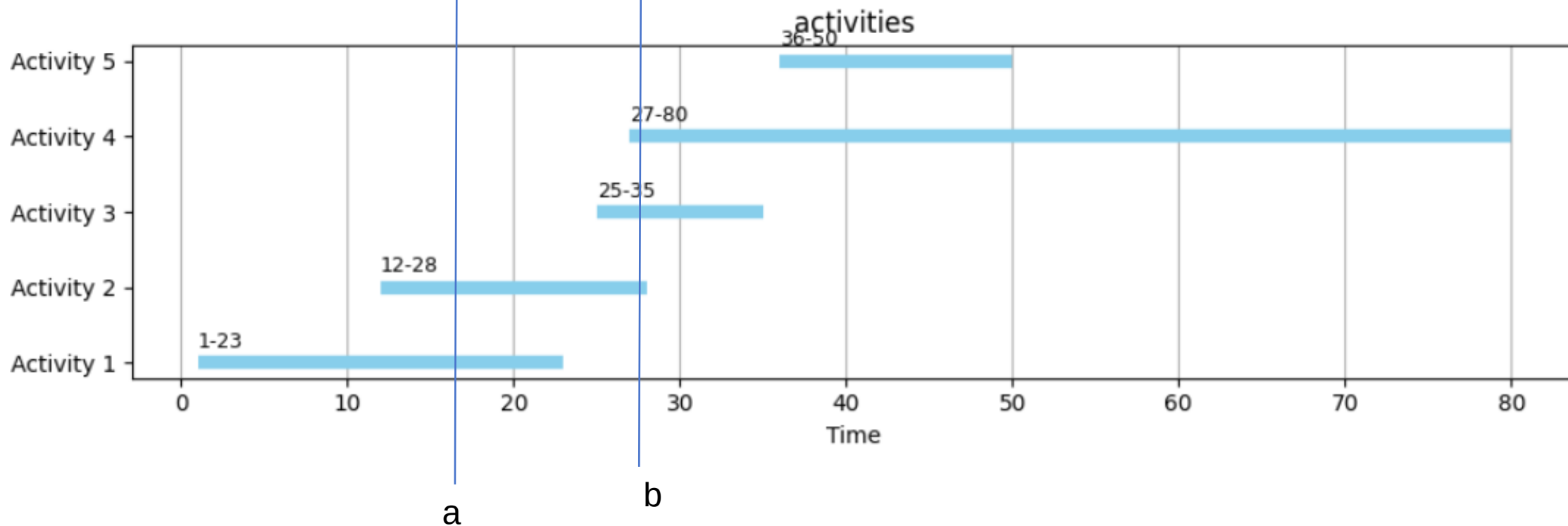
数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 k ，表示有 k 个待安排的活动。接下来的 k 行中，每行有 2 个正整数，分别表示 k 个待安排的活动开始时间和结束时间。时间以 0 点开始的分钟计。

结果输出：将计算的最少会场数输出到文件 output.txt。

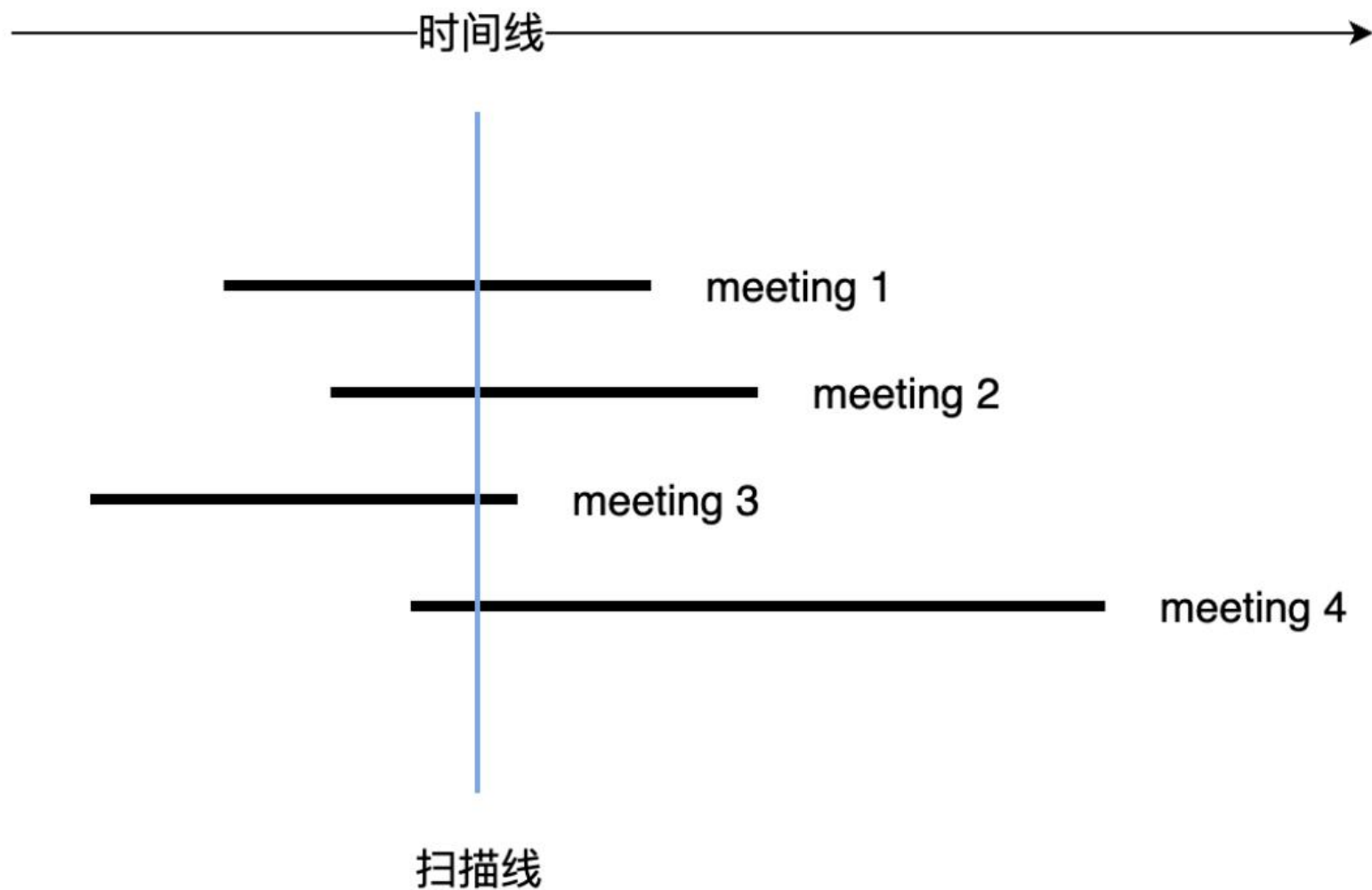
某一时刻同时进行的会议（活动）数量

a 竖线： $t=a$ 的时刻，最少要有2个不一样的会议室才能满足2个会议同时开

b 竖线： $t=b$ 的时刻，最少要有3个不一样的会议室才能满足3个会议同时进行。



问题等价于：同时进行的会议的最大数量



方法1

分析与解答：具体算法描述如下。

```
int greedy(vector<point> x) {
    int sum=0, curr=0, n=x.size();
    sort(x.begin (),x.end ());
    for(int i=0; i < n; i++) {
        if(x[i].leftend())
            curr++;
        else
            curr--;
        if((i == n-1 || x[i] < x[i+1]) && curr > sum)
            sum=curr;
    }
    return sum;
}
```

curr 当前同时进行的会议（活动）的数量

```
typedef struct Pointer {
    int time;
    is_leftend int;
}
```

leftend: 是否是开始时间点

// 处理 x[i]=x[i+1]的情况

方法2

贪心策略：如果有会议室空闲则当前活动使用该会议室，否则再新开一个会议室

```
▼ func minMeetingRooms(intervals [][]int) int {  
    allocator := &minHeap{}  
    heap.Init(allocator)  
  
    // 根据开始时间排序会议  
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })  
    heap.Push(allocator, intervals[0][1])  
  
    // 遍历剩余会议  
    for i := 1; i < len(intervals); i++ {  
        // 如果最早应该腾出的房间是空闲的，则将该房间分配给本次会议。  
        if intervals[i][0] >= (*allocator)[0] {  
            heap.Pop(allocator)  
        }  
        // 如果要分配一个新房间，那么我们也要添加到堆中，  
        // 如果分配了一个旧房间，那么我们还必须添加到具有更新的结束时间的堆中。  
        heap.Push(allocator, intervals[i][1])  
    }  
  
    // 堆的大小告诉我们所有会议所  
    return len(*allocator)  
}
```

优先队列维护每个会议室中的活动的最后结束时间

4-15 最优分解问题。

问题描述：设 n 是一个正整数。现在要求将 n 分解为若干互不相同的自然数的和，且使这些自然数的乘积最大。

算法设计：对于给定的正整数 n ，计算最优分解方案。

数据输入：由文件 input.txt 提供输入数据。文件的第 1 行是正整数 n 。

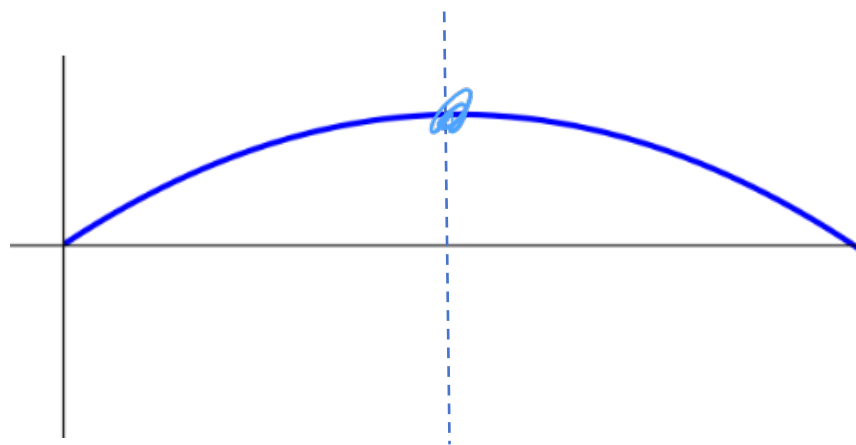
结果输出：将计算的最大乘积输出到文件 output.txt。

$n \leq 3 \Rightarrow \text{Max}$

1. 分解尽量不要包含1
2. 尽量分解成连续的自然数
3. 若分解中不包含1则尽量分解（如5能分解为 2,3, 不包含1, 尽量分解。）

两个数和为定值时，两数差距越小乘积越大：

a、n-a 乘积 $S = -a^2 + an$ ：



$$a=n/2, \quad a=n-a$$

扩展到多个数同样适用：

分解成尽量连续的自然数

```
void dicomp() {  
    k = 1;  
    if(n < 3) {  
        a[1] = 0;  
        return;  
    }  
    if(n < 5) {  
        a[k] = 1;  
        a[++k] = n-1;  
        return;  
    }  
}
```

```
    a[1] = 2;  
    n -= 2;  
    while(n > a[k]) {  
        k++;  
        a[k] = a[k-1]+1;  
        n -= a[k];  
    }  
    if(n == a[k]) {  
        a[k]++;  
        n--;  
    }  
    for(int i=0; i < n; i++)  
        a[k-i]++;  
}
```